INDIVIDUAL PROJECT REPORT

ON

"CampusConnect: Student Academic & Communication Portal"

INDIVIDUAL CONTRIBUTION REPORT — COMMUNITIES & CHAT MODULE

Submitted in partial fulfillment of the requirement for the award of Degree of

Bachelor of Technology in Computer Science & Engineering

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted by

AAYUSHMAN PATHAK

Enrollment No.: 24EARCS003

Individual responsibility: Communities & Chat Module

Under the guidance of

Er. Ram Babu Buri

Department of Computer Science & Engineering

Session: 2026–27

ARYA COLLEGE OF ENGINEERING & I.T.

SP-42, RIICO Industrial Area, Kukas, Jaipur

CERTIFICATE

This is to certify that the individual project report entitled “CampusConnect: Student Academic & Communication Portal — Communities & Chat Module” has been prepared by Aayushman Pathak (24EARCS003), a student of B.Tech Computer Science & Engineering at ARYA COLLEGE OF ENGINEERING & I.T., Jaipur, affiliated with Rajasthan Technical University, Kota, during session 2026–27.

The report documents the student's individual contribution to the Communities & Chat area of the group project. Final implementation evidence and certificate wording should be reviewed by the department before printing.

Role	Name	Signature
Project Guide	Er. Ram Babu Buri	_____
Head of Department	Dr. Akhil Pandey	_____
Student	Aayushman Pathak (24EARCS003)	_____

Date: _____ Place: Jaipur

DECLARATION

I, Aayushman Pathak (24EARCS003), declare that this report describes my contribution to the Communities & Chat module of CampusConnect. It is based on the available project history and the structure of the supplied example reports.

Implementation details, test results, diagrams, and module status must be checked against the final source code and running application before submission. Unverified behaviour is labelled as requiring verification. This report describes an individual contribution within a collaborative project and does not claim sole authorship of the complete system.

Aayushman Pathak
Enrollment No.: 24EARCS003
Date: _____

ACKNOWLEDGEMENT

I express sincere gratitude to my project guide, Er. Ram Babu Buri, and Head of Department, Dr. Akhil Pandey, for academic guidance and support. I thank the faculty members and teammates for their collaboration across frontend, backend, database, and feature modules.

Working on Communities & Chat provided practical experience with authenticated communication, conversation membership checks, REST APIs, real-time messaging, community interactions, and the importance of testing access-control behaviour.

ABSTRACT

CampusConnect is a web-based Student Academic & Communication Portal intended to bring academic and communication workflows into a common institutional interface. This individual report focuses on the Communities & Chat module, supporting direct conversations and structured community discussions.

The recorded development history includes authenticated conversation and message APIs, one-to-one find-or-create behaviour, paginated message retrieval, membership authorization, sender identity derived from the authenticated principal, duplicate-message protection using a client message identifier, and STOMP/WebSocket communication with SockJS fallback. The Community & Feedback Hub history includes post, comment, vote, and report concepts, database-backed services, and frontend components for feed and post details.

The report explains requirements, architecture, data concepts, security, UML models, test scenarios, integration workflow, challenges, and future improvements. Exact schema, endpoint paths, test counts, and final behaviour should be verified against the current repository before submission.

TABLE OF CONTENTS

- **Certificate**
- **Declaration**
- **Acknowledgement**
- **Abstract**
- CHAPTER 1 — INTRODUCTION
- CHAPTER 2 — PROJECT SCOPE AND RESPONSIBILITIES
- CHAPTER 3 — TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT
- CHAPTER 4 — REQUIREMENTS ANALYSIS
- CHAPTER 5 — SYSTEM ARCHITECTURE AND DESIGN
- CHAPTER 6 — CHAT MODULE DESIGN AND IMPLEMENTATION
- CHAPTER 7 — COMMUNITY & FEEDBACK HUB
- CHAPTER 8 — UML AND DIAGRAMS
- CHAPTER 9 — DATABASE AND API DESIGN
- CHAPTER 10 — SECURITY ARCHITECTURE
- CHAPTER 11 — IMPLEMENTATION WORKFLOW AND VERSION CONTROL
- CHAPTER 12 — TESTING STRATEGY AND TEST CASES
- CHAPTER 13 — RESULTS, LIMITATIONS AND VERIFICATION STATUS
- CHAPTER 14 — CHALLENGES AND LEARNING OUTCOMES
- CHAPTER 15 — FUTURE ENHANCEMENTS AND CONCLUSION
- References
- Appendix A — Final Verification Checklist
- Appendix B — Demonstration Checklist

LIST OF ABBREVIATIONS

Term	Meaning
API	Application Programming Interface
DTO	Data Transfer Object
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
JWT	JSON Web Token
MVC	Model–View–Controller
ORM	Object–Relational Mapping
REST	Representational State Transfer
RBAC	Role-Based Access Control
SPA	Single-Page Application
SQL	Structured Query Language
STOMP	Simple Text Oriented Messaging Protocol
UI/UX	User Interface / User Experience
WebSocket	Full-duplex communication protocol

CHAPTER 1 — INTRODUCTION

1.1 Project Overview

CampusConnect is a web-based Student Academic & Communication Portal intended to bring common campus workflows into a consistent interface. The overall group project includes academic information, announcements, profiles, clubs and events, direct communication, and a student community area. This report concentrates on Aayushman Pathak's individual contribution to the Communities & Chat module, while acknowledging that the application is developed collaboratively.

1.2 Problem Statement

Students commonly coordinate through multiple chat groups and separate channels. Important discussions can become difficult to find, conversations are disconnected from campus information, and community feedback may lack a structured place. CampusConnect's Communities & Chat module aims to provide a more organized environment for direct conversations and community discussions, with access controls and persistent records.

1.3 Objectives

The objectives of the module are to support authenticated conversations and messages; provide a community feed for posts and discussion; allow users to comment and vote where supported; provide reporting and moderation workflows; maintain chronological ordering; prevent unauthorized conversation access; and integrate the module into the shared CampusConnect frontend and backend.

1.4 Individual Contribution Boundary

The focus of this report is the Communities & Chat work associated with Aayushman Pathak. Development history records chat REST APIs, conversation/message handling, access control, WebSocket/STOMP support, client-side message handling, and an evolution of the chat area into a Community & Feedback Hub. The wider project's authentication foundation, announcements, academics, and Clubs & Events are team modules and are not claimed here as individually authored. Exact final behaviour should be checked against the current repository and final application.

CHAPTER 2 — PROJECT SCOPE AND RESPONSIBILITIES

2.1 Overall Project Context

CampusConnect is a modular group project. Each feature area is expected to connect through shared routing, backend API conventions, authentication, and database persistence. Communities & Chat complements academic and campus information by supporting user-to-user communication and user-generated discussion.

2.2 In-Scope Responsibilities

The scope covered in this report includes: conversation and message API behaviour; one-to-one conversation lookup or creation; pagination of message history; membership checks; identity derived from the authenticated principal; duplicate-message protection using a client message identifier; STOMP/WebSocket messaging with SockJS fallback; frontend hooks and REST fallback; community posts, comments, votes, reports, and moderation integration; feed ordering and loading behaviour; and integration/testing work.

2.3 Out-of-Scope Responsibilities

This report does not claim sole ownership of the entire CampusConnect application, the authentication module, academic/assignment workflows, announcements, Clubs & Events, deployment infrastructure, or all database design. Those areas are shared or owned by other team members. The report distinguishes the module interface from the internal implementation of other modules.

2.4 Contribution Matrix

Chat: conversation/message workflows, membership-based access, real-time messaging integration, and client-side message state. Community: post feed and discussion workflows, comments/voting/reporting design, moderation integration, ordering and loading improvements. Integration: API compatibility, branch/PR workflow, regression checks, and debugging. Shared work: project navigation, common authentication and database configuration are team-level dependencies.

CHAPTER 3 — TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT

3.1 Frontend

The CampusConnect frontend uses React with JavaScript and Vite. React components represent the feed, post details, conversation list, message pane, message bubbles, and dialogs. Hooks manage data retrieval and interaction state. The Vite development server supports local iteration and the production build checks whether the application bundles successfully.

3.2 Backend

The backend is implemented in Java using Spring Boot and Maven. The module follows a layered approach involving controllers, services, repositories, DTOs, and persistence entities. REST endpoints support operations that do not require a persistent socket connection; WebSocket/STOMP is used for real-time conversation updates where configured.

3.3 Database and Persistence

The project uses a relational database with Spring Data JPA/Hibernate in the backend. Community-related records include posts, comments, votes, and reports; chat records include conversations and messages. The actual schema and deployed database should be verified before submission, particularly where schema updates or environment configuration may differ between developers.

3.4 API and Real-Time Communication

REST APIs provide authenticated access to conversations and message history. STOMP destinations support real-time updates, while SockJS fallback and REST refresh paths improve compatibility where WebSocket connectivity is unavailable. A WebSocket subscription interceptor checks conversation membership before a client can subscribe to a private conversation destination.

3.5 Development and Collaboration Tools

The development workflow uses Git and GitHub feature branches and pull requests. The working branch associated with this contribution is ayushman-feature. The repository is https://github.com/Adit841/CampusConnect. Commits and merged PRs provide evidence of code history, but the exact final branch and commit should be checked at submission time.

CHAPTER 4 — REQUIREMENTS ANALYSIS

4.1 Functional Requirements

FR-01: an authenticated user can access permitted conversations. FR-02: users can find or create a one-to-one conversation. FR-03: a conversation's message history can be retrieved in pages. FR-04: users can send messages with sender identity taken from the authenticated principal. FR-05: duplicate client submissions can be detected using a client message identifier where supported. FR-06: authorized participants can receive real-time updates. FR-07: community users can browse posts. FR-08: community users can create posts and participate through comments or votes where enabled. FR-09: users can report content. FR-10: moderation actions are restricted to authorized administrators.

4.2 Non-Functional Requirements

Security: private conversations must be protected by membership checks. Reliability: message delivery and feed loading should handle failures clearly. Maintainability: controllers, services, repositories, DTOs, and UI components should have distinct responsibilities. Usability: loading, empty, and error states should be understandable. Performance: message history should be paginated and community queries should avoid unnecessary repeated database calls. Compatibility: frontend API paths and authentication headers must match backend configuration.

4.3 Actors and Permissions

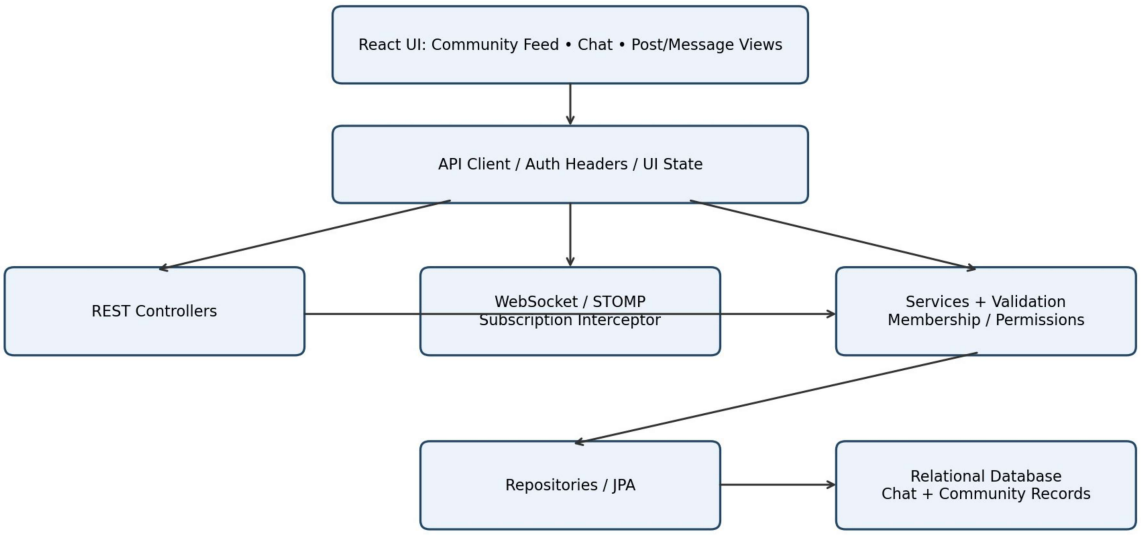
Student/user: browse community content, create posts, comment, vote, report content, and access conversations of which they are a member. Moderator/administrator: review reports and perform authorized moderation actions if the relevant backend permissions are enabled. Unauthenticated visitor: should not access protected conversation data and may receive an authentication response for protected community operations.

4.4 Constraints and Assumptions

The module depends on a working authentication system, matching frontend/backend URLs, database connectivity, and correct runtime configuration. The UI alone is not a security boundary: the backend must enforce authorization. Features and test counts mentioned in development notes are not treated as independently verified results in this report.

CHAPTER 5 — SYSTEM ARCHITECTURE AND DESIGN

Figure 5.1: Layered architecture of the Communities & Chat module



5.1 Layered Architecture

The frontend presentation layer renders the community feed and chat UI. An API/client layer sends HTTP requests and handles authentication. Controllers expose endpoints; service classes apply validation and business rules; repositories persist records; and the relational database stores conversations, messages, and community entities. WebSocket configuration and interceptors govern real-time delivery and protected subscriptions.

5.2 Chat Request Flow

A user selects or opens a conversation. The frontend requests the conversation and message history through the API client. The backend authenticates the request, checks membership, and retrieves the appropriate records. When a message is sent, the backend validates the request, determines the sender from the authenticated principal, persists the message, and broadcasts an update to authorized subscribers. The client reconciles the server response with local state to reduce duplicate display.

5.3 Community Request Flow

The community page requests a feed. The controller delegates to the service, which retrieves posts and associated interaction data from persistence. The frontend displays loading, empty, error, and populated states. Post creation and interaction operations are sent through API endpoints; server-side validation and authorization determine whether an operation is accepted.

5.4 Design Principles

Use server-side membership and role checks; do not trust a sender identifier supplied by the client; paginate message history; avoid duplicate message insertion; separate data transfer objects from persistence entities; provide clear loading and failure states; and keep unrelated module ownership boundaries intact.

CHAPTER 6 — CHAT MODULE DESIGN AND IMPLEMENTATION

6.1 Module Purpose

The chat module provides authenticated direct conversations inside CampusConnect. It is designed to keep conversations private to their participants while supporting message history and near-real-time updates.

6.2 Conversation Management

The recorded implementation includes conversation retrieval and one-to-one find-or-create behaviour. Conversation lists can be ordered by recent activity so that active discussions are easier to locate. The service layer is responsible for checking membership and retrieving data for the authenticated user.

6.3 Message Handling

Message APIs support retrieving messages with pagination and sending messages to a conversation. The sender identity is derived from the authenticated principal rather than trusting a sender field supplied by the browser. A client message identifier can be used to identify repeated submissions and reduce duplicate messages during retries or reconnects.

6.4 Real-Time Updates

The implementation history records STOMP over WebSocket under the /ws endpoint, SockJS fallback, and a JWT-aware WebSocket interceptor. The subscription interceptor checks that the user belongs to the conversation before allowing subscriptions to /topic/conversations/{id}. A REST fallback is useful when socket delivery is unavailable or reconnecting.

6.5 Frontend Components

The known frontend component areas include the conversation list, message pane, message bubble, new-conversation dialog, API client, and hooks for conversations/messages. The exact filenames and behaviour should be verified in the final repository. The UI should reconcile optimistic or pending messages with persisted server messages and keep conversation previews in recent-activity order.

6.6 Error Handling and Edge Cases

Important edge cases include invalid or expired tokens, attempts to open a conversation without membership, empty message history, duplicate submissions, network loss, WebSocket disconnect/reconnect, and inconsistent ordering after a new message. Clear user-facing feedback and server-side authorization are required for reliable behaviour.

CHAPTER 7 — COMMUNITY & FEEDBACK HUB

7.1 Purpose

The community area evolves the communication experience beyond private chat. It provides a structured feed for campus discussion and feedback, allowing users to publish posts and interact with community content.

7.2 Core Data Concepts

The development history records persistence entities for CommunityPost, CommunityComment, CommunityVote, and CommunityReport, with corresponding repositories, services, controllers, and DTOs. These represent the central post, its comments, user votes, and content reports. Actual field definitions should be checked against the current source and database schema.

7.3 Feed and Interactions

The frontend areas include CommunityFeed, CreatePostModal, PostDetailModal, and a community API service. Typical flows include retrieving the feed, opening a post, creating a post, commenting, voting, and reporting. Operations should be validated on the server, and the frontend should refresh or update affected content after a successful response.

7.4 Moderation and Permissions

The module includes a reported-content and moderation concept. Sensitive actions such as resolving reports, deleting content, and viewing moderation logs should be protected by server-side authorization. Public registration must not allow a user to grant themselves an administrator role. The final permissions and UI must be verified against the current build.

7.5 Feed Ordering and Performance

Development discussions identified incorrect ordering of recent posts and slow or persistent skeleton loaders. The intended solution is consistent timestamp-based sorting, efficient retrieval of related interaction data, removal of redundant loading states, and clear handling of empty/error responses. Any performance improvement should be measured against the current implementation before numeric claims are included.

CHAPTER 8 — UML AND DIAGRAMS

Figure 8.1: Use-case model

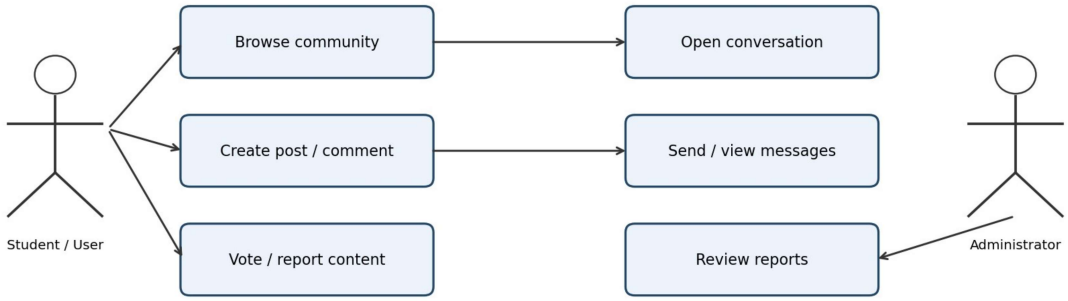


Figure 8.2: Chat message sequence

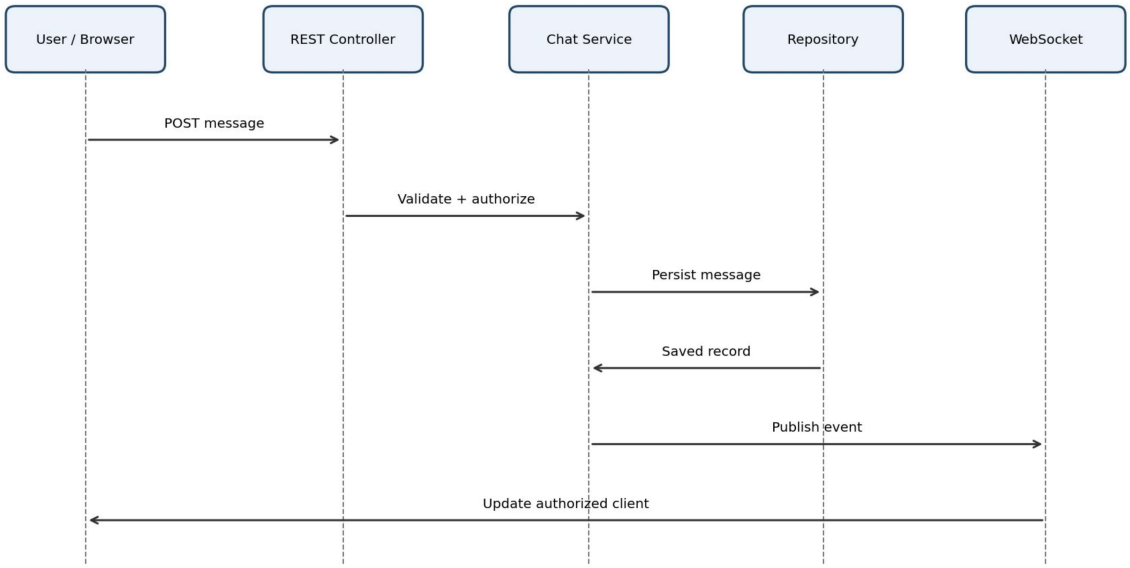
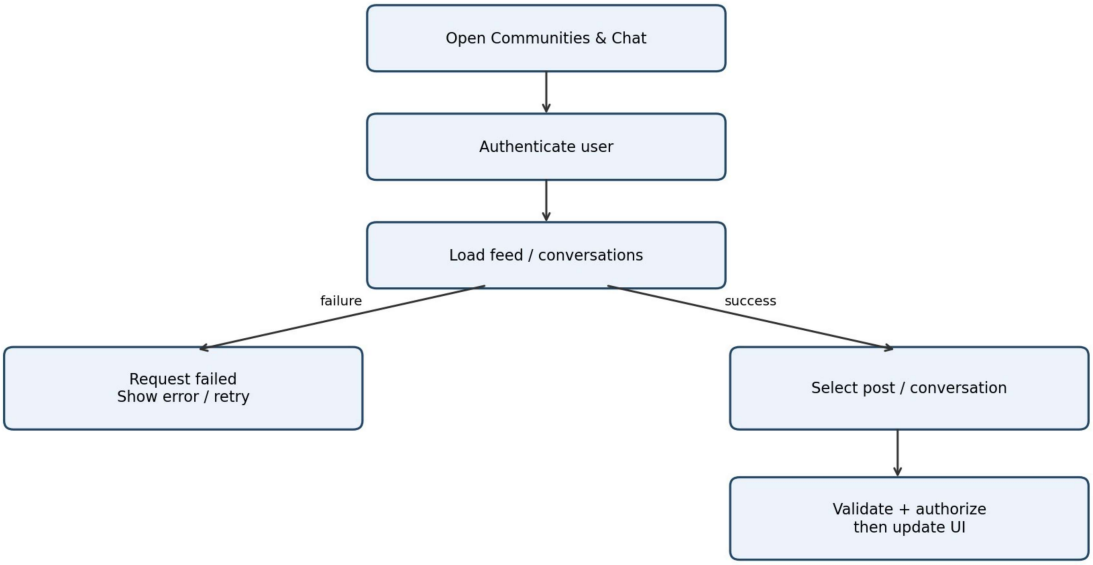


Figure 8.3: Activity flow



8.1 Use-Case Model

The use-case model includes the student/user, administrator, and CampusConnect system. A user can sign in, browse community posts, create a post, comment, vote, report content, open conversations, send messages, and view message history. An administrator can review reports and perform permitted moderation actions. Authentication and authorization are preconditions for protected operations.

8.2 Class / Domain Model

The conceptual domain model includes User, Conversation, Message, CommunityPost, CommunityComment, CommunityVote, and CommunityReport. A conversation has multiple participants and messages; a post can have multiple comments, votes, and reports. The exact entity mappings, key fields, and relationship cardinalities must be confirmed from the source before the final printed version is treated as a schema specification.

8.3 Sequence Model

The message sequence begins with the authenticated user sending a message from the browser. The API/controller validates the request, the service checks conversation membership, the repository persists the message, and the real-time layer publishes an event to the conversation topic. Only authorized subscribers should receive the event. If socket delivery fails, the client can refresh through REST.

8.4 Activity Model

A user opens the module, authentication is checked, the feed or conversation list loads, the user selects a post or conversation, and the relevant operation is submitted. The system validates the operation and permission. On success, the UI updates; on failure, it presents an error and preserves a recoverable state.

CHAPTER 9 — DATABASE AND API DESIGN

9.1 Chat Data Model

Conceptually, Conversation groups messages and identifies participants; Message belongs to one conversation and has a sender identity, content, timestamp, and optional client identifier. The actual schema may use join tables or separate participant records. This report intentionally avoids asserting column names that have not been checked against the final database schema.

9.2 Community Data Model

CommunityPost stores the primary discussion record; CommunityComment references a post; CommunityVote associates a user with a vote on a post; CommunityReport records a report and its status. Referential integrity and suitable indexes support consistent relationships and efficient retrieval. Exact columns and uniqueness constraints should be verified in the entity classes and schema.

9.3 API Design Principles

REST endpoints should use consistent request and response formats, validate incoming fields, return appropriate HTTP status codes, and avoid exposing internal entity structures unnecessarily. Protected endpoints should derive the user identity from the security context. Message history should be paginated and feed retrieval should avoid excessive per-post queries.

9.4 Authentication and Authorization

Authentication establishes the user identity; authorization determines what that user may access. A valid token does not automatically grant access to every conversation. Conversation membership must be checked for both REST access and WebSocket subscriptions. Administrative operations require a server-enforced role or permission check.

CHAPTER 10 — SECURITY ARCHITECTURE

10.1 Conversation Privacy

Private chat content is sensitive within the application. Every conversation read, message send, and subscription should validate membership. Guessing a conversation identifier must not allow another user to access the messages.

10.2 Trusted Identity

The backend should derive the sender from the authenticated principal. Accepting a client-provided sender ID without verification can permit impersonation. The service layer should also validate conversation existence, membership, message content, and request limits where applicable.

10.3 WebSocket Security

WebSocket handshakes and STOMP subscriptions require authentication and authorization. The subscription interceptor recorded in the project history validates conversation membership before a subscription to a private conversation topic is accepted. This check complements, rather than replaces, REST authorization.

10.4 Community Moderation Security

Post reporting and moderation must not rely on hiding buttons in the frontend. The backend should verify privileges for reading reports, resolving them, deleting content, and accessing moderation logs. Credentials, JWT signing keys, and database passwords must remain outside version control.

CHAPTER 11 — IMPLEMENTATION WORKFLOW AND VERSION CONTROL

11.1 Branch-Based Development

The development workflow uses a feature branch named ayushman-feature and a shared GitHub repository. Branches allow work to be developed and tested before integration. Pull requests provide a reviewable record of changes and help prevent direct, unreviewed modifications to the integration branch.

11.2 Recorded Milestones

The available project history records commit 1f0eb39 for chat controller tests and hardening the useChat hook, and commit 730836e titled “feat(community): evolve chat module into Campus Community & Feedback Hub”. Other later commits and merges may exist; the final report should be checked against GitHub for the exact final SHA and merged state.

11.3 Integration Practices

Before merging, inspect the diff, confirm that only intended files are included, run backend tests and the frontend production build, and resolve conflicts without discarding teammate work. Environment files containing credentials must remain untracked. A clean local branch alone does not prove that all intended commits are pushed or merged.

11.4 Repository Reference

Project repository: https://github.com/Adit841/CampusConnect. Use the repository and the relevant commit or pull request as implementation evidence. The URL alone does not prove that every described feature was merged or passed testing; check the relevant source and CI/test results before final submission.

CHAPTER 12 — TESTING STRATEGY AND TEST CASES

12.1 Testing Objectives

Testing should verify functional correctness, access control, persistence, failure handling, and the integration between the frontend and backend. Tests should cover both expected user flows and security-sensitive negative cases.

12.2 Chat Test Cases

TC-01: authorized member retrieves a conversation. TC-02: non-member cannot retrieve a conversation. TC-03: message history is returned in the expected order and page size. TC-04: sender identity is taken from authentication. TC-05: repeated client message ID does not create an unintended duplicate. TC-06: authorized member can subscribe to the conversation topic. TC-07: non-member subscription is rejected. TC-08: a disconnected client can recover message history through REST.

12.3 Community Test Cases

TC-09: feed endpoint returns permitted posts. TC-10: post creation validates required fields. TC-11: comment and vote operations enforce authentication. TC-12: reports are persisted and associated with the intended post. TC-13: non-admin users cannot resolve reports or access moderation logs. TC-14: the feed sorts posts consistently by the intended timestamp. TC-15: frontend shows loading, empty, error, and populated states correctly.

12.4 Evidence and Result Reporting

The project history contains references to backend test runs and frontend production builds reported by the development agent. This report does not independently rerun those tests, so exact pass counts are intentionally omitted. Before submission, execute the current test suite and build, capture genuine results, and add screenshots or CI evidence where available.

CHAPTER 13 — RESULTS, LIMITATIONS AND VERIFICATION STATUS

13.1 Recorded Outcomes

The available history records the development of authenticated chat APIs, conversation membership checks, message pagination, WebSocket/STOMP integration, a community-feed implementation, post-related persistence concepts, and work on feed ordering and loading behaviour. These are development-history observations and should be cross-checked against the current repository.

13.2 Known Integration Risks

A teammate reported that Clubs & Events and the Community Feed did not load on their machine while the local version worked. A screenshot showed a 401 Unauthorized response. This demonstrates that environment and authentication differences can affect the experience even when a page renders locally. The exact endpoint and root cause must be confirmed from the browser Network panel and backend logs.

13.3 Limitations

The module depends on the shared authentication system, correct database configuration, and the integration branch. Real-time behaviour can be affected by connectivity and reconnect handling. Moderation workflows require careful authorization. The final application state may differ from earlier local screenshots or agent-reported test results.

13.4 Verification Checklist

Before final submission, verify the latest main branch; inspect the actual entities and endpoint paths; run backend tests and the frontend build; test authorized and unauthorized users; check the chat WebSocket subscription; verify community feed ordering; confirm report/moderation permissions; and capture screenshots of working features. Replace any planned or unverified description with the actual result.

CHAPTER 14 — CHALLENGES AND LEARNING OUTCOMES

14.1 Authorization Complexity

A key lesson is that authentication and authorization are different. Knowing who the user is does not prove they belong to a particular conversation. Membership checks must be performed at the service or security boundary for every protected path.

14.2 Real-Time State Synchronization

Real-time interfaces must reconcile local state, REST responses, socket events, retries, and reconnects. Duplicate message identifiers and consistent ordering help prevent duplicate display and confusing timelines.

14.3 Feed Loading and Ordering

A community feed needs stable timestamp ordering, efficient data retrieval, and meaningful loading/error states. A spinner that never disappears can be caused by a failed request, stale state, or an exception—not merely slow rendering—so diagnosis should follow the network request and component state.

14.4 Collaboration and Debugging

Working with a shared repository highlighted the importance of clear module ownership, branch discipline, descriptive commits, small pull requests, and testing after integration. Local success should be checked against the teammate’s environment when failures occur across machines.

14.5 Learning Outcomes

The work connects frontend component design with backend services, REST APIs, relational persistence, authentication context, WebSocket communication, authorization, test design, and Git-based collaboration. It also reinforces the value of reporting verified results honestly and distinguishing implemented functionality from planned work.

CHAPTER 15 — FUTURE ENHANCEMENTS AND CONCLUSION

15.1 Future Enhancements

Potential improvements include message read receipts, typing indicators, attachment support with safe file validation, search across conversations, richer notification controls, moderation audit trails, report filters, accessibility refinements, pagination and indexing improvements, and stronger automated end-to-end tests. Each improvement should be prioritized against actual user needs and project constraints.

15.2 Conclusion

The Communities & Chat module contributes communication and community discussion capabilities to CampusConnect. Its design combines authenticated REST operations, conversation membership checks, message persistence, real-time delivery, and community interactions. The report describes the recorded development direction while making verification requirements explicit. Final submission should be supported by the current repository, test output, and screenshots of the running application.

REFERENCES

1. 1. CampusConnect project repository, https://github.com/Adit841/CampusConnect (accessed for project context; verify current branches and commits before submission).
2. 2. React documentation, https://react.dev/ — component-based user interface development.
3. 3. Vite documentation, https://vite.dev/ — frontend development and production build tooling.
4. 4. Spring Boot documentation, https://spring.io/projects/spring-boot — Java application framework.
5. 5. Spring Framework WebSocket/STOMP documentation, https://docs.spring.io/spring-framework/reference/web/websocket.html.
6. 6. Spring Data JPA documentation, https://spring.io/projects/spring-data-jpa.
7. 7. MDN Web Docs, Fetch API and WebSocket API, https://developer.mozilla.org/.
8. 8. Git documentation, https://git-scm.com/doc — version control workflow.